

Pedro Luis Lobato Barros

Core Logic Lead & Production Ownership

Bogotá D.C., Colombia

Remote-first

+57 318 507 9105

✉ pedro.lobato2610@gmail.com

in [lobaton2610](#)

🔗 [Lob26](#)

Profile

Owner of the three engines a high-growth Fintech and PropTech business runs on — property valuation, client screening and contract generation — and of how they behave in production. Cut operational wait times by up to **96%**, caught and repelled a live attack on production, and bridged backend systems (Python, Go, Ruby) with CRM ecosystems (Salesforce). Sole engineer on every core engine I have built: I grow by taking on harder systems, not larger teams. Fluent across English and Spanish with Legal, Commercial and Operations.

Systems that stopped needing me. Credentialing code still running with minimal maintenance two years after launch · a contract engine issuing every contract the company signs while sitting in maintenance · a backend posture that has held as attack volume grew. I measure the work by what it does after I stop touching it.

Technical Toolbox

Languages	Python, Go, Ruby, TypeScript, Solidity, JavaScript, SQL
Integrations	Salesforce (Apex, REST), Slack, RabbitMQ, REST APIs, custom AWS & Slack wrappers
Databases & Data	PostgreSQL, BigQuery, SOQL/SOSL
DevOps & Cloud	AWS, GCP, Docker, CI/CD, GitHub Actions
Practices	SRE principles, security architecture, system design, code review
Languages spoken	English (C1) · Spanish (Native)

Experience

Aug 2024 – Present **Software Engineer — Core Logic Lead, Duppla, Bogotá, Colombia, *Technical ownership***
· Cross-functional lead

Translator between engineering and the commercial, legal and operations teams — and the engineer who then builds what the conversation agreed on.

- **Core Product Ownership (–96%)** [*Commercial · Operations*]

Rebuilt the Real Estate Evaluator over four iterations: full backend automation, then a human-in-the-loop review layer, then a migration off Retool once low-code became the constraint rather than the shortcut, then an end-to-end pass over back and front. Analyst time per property went 90 minutes to 15 to 10 to 4 on the happy path. The engine sustains 1,100 automated evaluations a day, and the 3 minutes an automated run still takes is 90 seconds of a third-party data provider, not our code.

- **Live Attack Defense (100% repelled)** [*Engineering · Leadership*]

Months before it mattered I had added a deliberately small detector — two heuristic buckets: HTTP exceptions carrying known attack signatures, and sources accumulating failed requests. When a bot exposed the full API surface and attacks hit billing and reporting within minutes, that cheap signal is what caught it live. I shipped endpoint-level defenses in real time and hardened the backend into a posture that has repelled every attempt since, as volume has grown.

- **Contract Automation (–90% review errors)** [*Legal · Commercial · Operations*]

Sole designer and owner of the contract generation engine — unowned work I picked up after making the case for it — now the only path through which the company issues contracts. Modelled the legal document as validated data instead of a text template: typed placeholders and validation rules refuse an incomplete contract before it reaches a lawyer. Review errors down 90%, request-to-approval turnaround down 40%. Load-tested to roughly fifteen times current daily volume in a single hour, where the ceiling is the document provider's rate limit rather than the pipeline — sized for two orders of magnitude of growth, not for today.

- **Salesforce Bridge** [*Commercial · RevOps*]

Not a Salesforce engineer — the engineer on the other side of the seam. I know the org's data model, SOQL/SOSL, Apex and the REST surface well enough to be the one proposing the schema and optimisation changes, and I contribute the integration side: data cleansing, shared utilities, and keeping CRM records consistent enough for the revenue team to trust them.

- **Engineering Leverage (–30% tech debt)** [*Engineering*]

Custom AWS and Slack wrappers now used by the whole team — four engineers, every backend — so nobody hand-rolls a client per service. The front-end architecture I designed for contract generation became the pattern the evaluator rebuild was based on. I also audit code I do not own and take the reliability defects I find there; new engineers ship on day one instead of deciphering.

- **Technical Foresight** [*Engineering · Leadership*]

When AI-generated internal front-ends started appearing, I argued at the third one that the pattern outruns ownership faster than anyone expects, and that the cost arrives later as risk nobody has scoped. The trajectory played out on the timeline I described. I also made the case that a design guide alone cannot steer generated code — without machine-readable context an agent builds a generic product, not yours. I raise this class of problem before it becomes a ticket.

Jun 2024 – **Software Development Intern, Duppla, Bogotá, Colombia, *First production ownership***

Aug 2024 Took the early leap from academic code into production work: backend services for customer assessment and internal logistics on Slack.

- **Business Process Migration (–90%)** [*Legal · Risk · Origination*]

Given one line of direction — “get this under 10 minutes” — I traced the commercial call flow to find which data actually gated a decision, then rebuilt the Client Evaluator backend around it instead of pre-warming everything. Thirty minutes of manual work collapsed into three. It has since absorbed a 1,500-evaluation day against a normal load of 170 without failing.

- **Internal Logistics** [*People Ops*]

Slack automation for attendance and office logistics still used daily by HQ.

- **Stability** [*IT*]

Credentialing code running with minimal maintenance two years post-launch — the quiet kind of reliability.

Education

2020 – 2027 **B.S. Systems & Computing Engineering**, *Universidad de los Andes*, Bogotá, Colombia
(Expected)

Notable Engineering Projects

Real Estate Evaluation Engine

-96% analyst time

[Commercial · Operations]

Human problem. Commercial analysts waited 90 minutes per property evaluation — long enough for a client to cool off and a negotiation to stall. I noticed the wait, asked about it, and was invited onto the project.

Technical answer. Four iterations, each answering the constraint the last one exposed. Full backend automation first; then a human-in-the-loop review layer on Retool; then a migration into our own intranet once low-code turned into a fixed-size box that could not absorb the requests coming in; then an end-to-end pass over back and front — events, caching, vectorised scoring — once building the contract engine taught me where the remaining time was hiding. Analyst time went 90 minutes to 15 to 10 to 4 on the happy path, 7 on a normal one. The engine sustains 1,100 automated evaluations a day and 100 human reviews per evaluator; an automated run is 3 minutes, 90 seconds of which belongs to a third-party data provider.

Business Client Evaluator

-90% cycle time

[Commercial · Origination]

Human problem. Origination sat on every prospect for 30 minutes of manual screening — good leads went cold in the queue while unfit ones ate the same half-hour before they could be discarded. The brief was one sentence: get it under 10 minutes.

Technical answer. I traced the commercial call flow first to find which data actually gated a decision, because the naive answer — pre-warm everything — breaks on the leads that reach the stage with fields missing. Full backend rewrite around that finding: new data model, leaner queries, asynchronous scoring. The cycle dropped from 30 minutes to 3, and the service has since absorbed a 1,500-evaluation day against a normal load of 170 without failing — if the top of the funnel widens, the engine is already there.

Contract Automation Engine

-90% review errors

[Legal · Commercial · Operations]

Human problem. Contract review was the bottleneck the whole business ran through: three separate teams all sent requests, and every contract was hand-crafted from templates riddled with typo-prone patterns and half-filled variable slots. An earlier “automation” had only put the data on one low-code page — no generation, no validation, and covering part of the flow — and was painful enough to work on that even small fixes stalled. Nobody owned it. I made the case for the urgency, got no takers, and picked it up.

Technical answer. Audited the business process and the data flow before writing anything, then built the real automation from scratch as the sole engineer. Every template rebuilt around typed placeholders, with a validation layer in front of generation so an incomplete or inconsistent contract is refused at the boundary rather than caught at a lawyer’s desk. It now issues 100% of the company’s contracts — versioned, validated, auditable — with review errors down 90% and request-to-approval turnaround down 40%. Load-tested in one hour to roughly fifteen times a normal day’s volume, bounded by the document provider’s rate limit rather than by the pipeline: sized for two orders of magnitude of growth, not for today. Stable enough since launch to sit in maintenance, owned by one engineer.

Live Attack Defense

100% repelled

[\[Engineering · Leadership\]](#)

Human problem. A bot slipped in a crafted request that revealed our full API surface; within minutes, attacks flooded the billing and reporting endpoints — the kind of hit that moves real money.

Technical answer. Months earlier, in my seventh month, I had added a deliberately small detector: raise on error, then sort into two heuristic buckets — HTTP exceptions carrying known attack signatures, and sources accumulating failed requests. It was not a rich feature, and it did not need to be; it is what caught the attack live. I shipped endpoint-level defenses on the targeted routes before they could execute and hardened the backend into a posture that has repelled every subsequent attempt as attack volume has grown.

Public  Fullstack Spring/Angular App |  JDO Data Persistence Project
Repositories